ALL 7 DOMAINS

Sample Questions

by Exam Domain

Databricks Certified Data Engineer Associate (exam version May 4, 2026)

- 22 scenario-based questions in the official exam-guide sample format
- Weighted by domain: D1 2 | D2 4 | D3 4 | D4 3 | D5 3 | D6 3 | D7 3
- Objective line quoted verbatim from the exam outline
- Answer key with explanations, distractor analysis, and doc URLs
- Claims checked against docs.databricks.com (Sep 2026)

Study-only material. Not affiliated with Databricks. Real exam items are not disclosed.

How to Use This Set

Each question copies the structure of the official exam guide samples: an **Objective** line quoted from the exam outline, a short scenario with concrete constraints, a one-sentence question, and four options (A to D) with exactly one correct answer. Answers and explanations are collected at the end, as in the exam guide, so you can attempt the questions without seeing them.

Section	Domain	Exam weight	Questions	Range
Section 1	Databricks Intelligence Platform	6%	2	Q1-Q2
Section 2	Data Ingestion and Loading	21%	4	Q3-Q6
Section 3	Data Transformation and Modeling	22%	4	Q7-Q10
Section 4	Working with Lakeflow Jobs	16%	3	Q11-Q13
Section 5	Implementing CI/CD	10%	3	Q14-Q16
Section 6	Troubleshooting, Monitoring, and Optimization	10%	3	Q17-Q19
Section 7	Governance and Security	15%	3	Q20-Q22

EXAM FOCUS Pacing: 45 scored questions in 90 minutes is about **2 minutes per question**. Time yourself on this set at the same pace.

TIP Pattern from the official samples: the correct option is usually the **Databricks-native, managed, or automatic** feature. Distractors tend to be (1) workable but sub-optimal manual fixes, (2) legacy or anti-pattern approaches, (3) swapped details such as formats, defaults, or limits, and (4) a real product applied to the wrong use case.

NOTE Product names follow the current exam guide: Lakeflow Jobs, Lakeflow Connect, Lakeflow Spark Declarative Pipelines, and Declarative Automation Bundles (formerly Databricks Asset Bundles).

SECTION 1 Databricks Intelligence Platform

Exam weight 6%

Question 1

Objective: *Understand the core components of the Databricks Data Intelligence Platform, such as its architecture, Delta Lake, and Unity Catalog.*

A scheduled MERGE job ran yesterday with a faulty join condition and overwrote values in about 40% of the rows of the Unity Catalog managed Delta table `sales.silver.orders`. Compliance needs to know who ran the operation and when, and the team must return the table to its pre-MERGE state without reloading from source. VACUUM runs weekly with default retention settings.

Which approach meets these requirements?

- **A.** Run `VACUUM orders RETAIN 0 HOURS` to remove the bad data files, then re-run the MERGE with the corrected condition
- **B.** Use `DESCRIBE HISTORY` to identify the MERGE version, user, and timestamp, then run `RESTORE TABLE ... TO VERSION AS OF` the version before the MERGE
- **C.** Use Unity Catalog lineage to roll the table back to a snapshot of its upstream source tables
- **D.** Create a `SHALLOW CLONE` of the table and drop the original, because clones retain the pre-MERGE data

Question 2

Objective: *Understand Databricks Data Intelligence Platform's compute services, including their characteristics, limitations, and cost models, and select the most suitable option for each workload use case.*

A team runs a nightly PySpark notebook ETL in a Unity Catalog-enabled workspace. They want fast startup, automatic scaling, no cluster configuration to maintain, and no charges for idle compute. The code is compatible with standard access mode.

Which compute option best meets these requirements?

- **A.** An all-purpose cluster left running 24/7 so the job never waits for startup
- **B.** Serverless compute for jobs
- **C.** A classic job cluster with 20 fixed workers and manually selected instance types
- **D.** A serverless SQL warehouse assigned to the notebook task that runs the PySpark code

SECTION 2 Data Ingestion and Loading

Exam weight 21%

Question 3

Objective: *Use the COPY INTO command to incrementally load files from cloud object storage (ADLS/S3/GCS) into Unity-Catalog-governed tables.*

A daily job runs `COPY INTO main.bronze.invoices FROM 's3://acme-landing/invoices/' FILEFORMAT = CSV`. The vendor found errors and re-uploaded corrected versions of 12 files **using the same file names**. The engineer re-runs the job, and 0 rows are loaded.

What explains this behavior, and how should the engineer load the corrected files?

- **A.** COPY INTO skips files it has already loaded, even if they were modified; use `COPY_OPTIONS ('force' = 'true')` for the reload
- **B.** COPY INTO only loads files newer than the table's last commit timestamp; update the files' modification time and re-run
- **C.** The table schema changed; set `COPY_OPTIONS ('mergeSchema' = 'true')`
- **D.** COPY INTO tracks progress in a checkpoint location; delete the checkpoint directory to reprocess all files

Question 4

Objective: *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity Catalog-governed tables.*

An Auto Loader stream ingests JSON files with `cloudFiles.schemaLocation` set and **no schema provided**. An upstream system adds a new field, `discount_code`. The team wants this column to appear in the bronze table.

What happens with the default schema evolution configuration?

- **A.** The new field is silently dropped, and the schema changes only if `schemaEvolutionMode` is explicitly set
- **B.** The stream fails with an `UnknownFieldException` after adding the new column to the schema, and restarting the stream resumes with the updated schema
- **C.** The stream keeps running and writes the new field only to the `_rescued_data` column
- **D.** The stream widens the data types of existing columns to accommodate the new field and continues

Question 5

Objective: *Prioritize between Auto Loader, Lakeflow Connect (standard and managed connectors), partner connectors, and other ingestion methods based on technical requirements such as data volume, ingestion frequency, data types, and governance needs with Unity Catalog.*

Marketing needs Salesforce objects landed daily into Unity Catalog tables. The data team is small and does not want to maintain custom API code. Loads must be incremental (only changed records), run on serverless compute, and be governed by Unity Catalog.

Which ingestion approach should the data engineer choose?

- **A.** A notebook that calls the Salesforce REST API, handles pagination, and MERGEs into Delta, scheduled with Lakeflow Jobs
- **B.** A nightly Salesforce CSV export to S3, ingested with Auto Loader
- **C.** A Lakeflow Connect managed connector for Salesforce
- **D.** A JDBC read from a notebook that pulls every full table each night

Question 6

Objective: *Use Auto Loader with schema enforcement and schema evolution in batch modes (for example, directory listing or file notification) to land data into Unity Catalog-governed tables.*

An Auto Loader stream in directory listing mode watches a landing path that now receives millions of small files per day. Each micro-batch spends more and more time discovering new files. The workspace admin can enable file events on the external location.

Which change improves file discovery scalability?

- **A.** Switch Auto Loader to file notification mode using file events
- **B.** Set `cloudFiles.maxFilesPerTrigger` to 1 so that each batch lists fewer files
- **C.** Replace Auto Loader with COPY INTO scheduled every minute
- **D.** Set `cloudFiles.schemaEvolutionMode` to `none` to reduce listing overhead

SECTION 3 Data Transformation and Modeling

Exam weight 22%

Question 7

Objective: *Perform data deduplication operations and aggregate operations on DataFrames, such as count, approximate count distinct, and mean, summary.*

The bronze `orders` table contains duplicate records for the same `order_id` because of upstream retries. The duplicates are identical except for the `ingest_ts` column. The silver table must contain exactly one row per `order_id`.

Which PySpark operation produces the required result?

- **A.** `df.distinct()`
- **B.** `df.dropDuplicates(["order_id"])`
- **C.** `df.groupBy("order_id").count()`
- **D.** `df.dropna(subset=["order_id"])`

Question 8

Objective: *Combine DataFrames with operations such as Inner join, left join, broadcast join, multiple keys, cross join, union, and union all.*

A job joins a 2 TB fact DataFrame `transactions` to an 80 MB dimension DataFrame `stores` on `store_id`. The physical plan shows a SortMergeJoin with a large shuffle of `transactions`. Executors have plenty of memory.

Which change most directly avoids shuffling the fact table?

- **A.** `transactions.join(broadcast(stores), "store_id")`
- **B.** `transactions.crossJoin(stores).filter("t.store_id = s.store_id")`
- **C.** `transactions.unionByName(stores, allowMissingColumns=True)`
- **D.** Increase `spark.sql.shuffle.partitions` to 2000

Question 9

Objective: *Understand the difference between, and how to build, Gold layer objects such as materialized views, views, streaming tables, and tables for BI and analytics teams in Unity Catalog.*

A BI dashboard shows daily revenue by region, aggregated from `silver.orders`. That table is batch-updated and includes **updates and deletes**. The dashboard needs low query latency, so results should be precomputed and refreshed incrementally when possible.

Which Gold layer object should the data engineer create?

- **A.** A standard view
- **B.** A materialized view
- **C.** A streaming table that reads from `silver.orders`
- **D.** A temporary view created in the dashboard's session

Question 10

Objective: *Apply data quality checks and validation rules to ensure reliable Silver and Gold datasets.*

In a Lakeflow Spark Declarative Pipeline, rows in the silver table with a null `customer_id` must be excluded. The pipeline must keep running, and the number of excluded rows must be visible in pipeline metrics.

Which expectation definition meets these requirements?

A. `CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL)`

B. `CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW`

C. `CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON VIOLATION FAIL UPDATE`

D. `ALTER TABLE silver ADD CONSTRAINT c CHECK (customer_id IS NOT NULL)` on the target table

SECTION 4 Working with Lakeflow Jobs Exam weight 16%

Question 11

Objective: *Choose between time-based and data-driven triggers based on data availability and pipeline dependencies.*

A Gold job is scheduled for 6:00 AM, but the upstream Unity Catalog managed table `silver.orders` finishes updating at any time between 3:00 and 8:00 AM, so the Gold job sometimes runs on stale data. The upstream team writes in several commits over about 10 minutes.

Which trigger configuration best fits this dependency?

- **A.** Keep the time-based schedule but move it to 9:00 AM
- **B.** A table update trigger on `silver.orders` with *Wait after last change* set to about 15 minutes
- **C.** A file arrival trigger on the storage location of `silver.orders`
- **D.** A continuous trigger so that the job is always running

Question 12

Objective: *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration*

The same ingestion notebook must run once for each of 25 regions. The list of regions is produced by an upstream task, `get_regions`, as a task value named `regions`. At most 5 regions should run in parallel.

How should the data engineer configure this?

- **A.** A For each task with Inputs `{{tasks.get_regions.values.regions}}` and Concurrency set to 5
- **B.** 25 separate notebook tasks copied from one another, each with a hard-coded region
- **C.** An If/else condition task that checks each region in turn
- **D.** Set the job's maximum concurrent runs to 5 and trigger the job 25 times through the API

Question 13

Objective: *Implement control flows (retries and conditional tasks such as branching and looping) using Lakeflow Jobs for pipeline orchestration*

A `validate` task sets the task value `bad_row_count`. If the value is greater than 0, the job must run `quarantine_report`. Otherwise it must run `publish_gold`.

Which Lakeflow Jobs feature implements this branching?

- **A.** An If/else condition task with `{{tasks.validate.values.bad_row_count}} > 0`, with `quarantine_report` on the true branch and `publish_gold` on the false branch
- **B.** Retries configured on the `validate` task
- **C.** Set *Run if* on `publish_gold` to *All done*
- **D.** A For each task that iterates over `bad_row_count`

SECTION 5 Implementing CI/CD Exam weight 10%

Question 14

Objective: *Understand environment-specific configuration using Automation Bundle (formerly Databricks Asset Bundles) variables and overrides while promoting the same codebase across dev, test, and prod targets.*

One bundle deploys the same job to dev, test, and prod. Each environment writes to a different catalog (`dev_catalog`, `test_catalog`, `prod_catalog`). The team wants a single job definition.

Which approach should the data engineer use?

- **A.** Declare a custom variable `catalog` with a default, reference `${var.catalog}` in the job, and override it under `targets..variables`
- **B.** Keep three copies of `databricks.yml`, one per environment branch
- **C.** Hard-code `prod_catalog` and rely on Unity Catalog permissions to block writes from dev
- **D.** Store the catalog name as a notebook widget default and edit it by hand before each deploy

---

Question 15

Objective: *Understand the Databricks CLI to validate, deploy, and manage Declarative Automation Bundles (formerly Databricks Asset Bundles) and other workspace assets in automated CI/CD workflows.*

A CI pipeline runs on every merge to `main`. It must catch bundle configuration errors **before** anything is deployed and then deploy the bundle to the prod target.

Which Databricks CLI sequence should the pipeline run?

- **A.** `databricks bundle validate -t prod`, then `databricks bundle deploy -t prod`
- **B.** `databricks bundle run -t prod`, then `databricks bundle validate`
- **C.** `databricks bundle init -t prod`, then `databricks jobs create`
- **D.** `databricks workspace import-dir` to upload notebooks, then create the job in the UI

Question 16

Objective: *Deploy Declarative Automation Bundles (formerly Databricks Asset Bundles) to package, configure, and promote Lakeflow Jobs, Lakeflow Spark Declarative Pipelines, and other workspace assets across dev, test, and prod environments.*

Several developers deploy the same bundle to a shared dev workspace. Their deployed jobs must not collide by name, and scheduled jobs in dev copies must not fire automatically.

Which bundle setting meets these requirements?

- **A.** Set `mode: development` on the dev target
- **B.** Set `mode: production` on the dev target
- **C.** Give each developer a separate Git folder, with no change to the bundle
- **D.** Remove the `schedule` block from `databricks.yml` before each deploy

SECTION 6 Troubleshooting, Monitoring, and Optimization

Exam weight 10%

Question 17

Objective: *Identify common performance bottlenecks such as data skew, shuffling, and disk spilling by interpreting stage-level metrics in the Spark UI.*

In the Spark UI, a slow stage's Summary Metrics show uniform task durations (Median 2.1 min, 75th percentile 2.3 min, Max 2.6 min). The stage details show **Spill (Memory) 180 GB** and **Spill (Disk) 45 GB**. `spark.sql.shuffle.partitions` has been set to 32 on a very large dataset.

Which change most likely reduces the stage runtime?

- **A.** Salt the join key to spread skewed keys across partitions
- **B.** Increase the number of shuffle partitions (or set `spark.sql.shuffle.partitions=auto`) so that each partition fits in memory
- **C.** Decrease `spark.sql.shuffle.partitions` to 8
- **D.** Disable adaptive query execution

Question 18

Objective: *Understand the features of Liquid Clustering and predictive optimization.*

A Delta table is partitioned by `event_date` and Z-ordered on `user_id`. Query patterns have shifted to filtering by `country` and `device_type`, and many partitions contain small files. The team wants to be able to change the layout keys later without a full rewrite, and ideally let Databricks choose the keys.

Which approach should the data engineer take?

- **A.** Add `country` and `device_type` as additional partition columns
- **B.** Migrate to liquid clustering, for example `CLUSTER BY AUTO` with predictive optimization enabled
- **C.** Keep the date partitioning and add liquid clustering on `country` to the same table
- **D.** Run `OPTIMIZE ... ZORDER BY` daily on all four columns

Question 19

Objective: *Identify trends in job performance using the Lakeflow Jobs run history view to compare current execution times against historical baselines.*

The team suspects a nightly job has been slowing down over the past month. They want to compare each run's total duration and each task's duration against previous runs, and to be warned automatically when a run exceeds 45 minutes.

Which approach meets these requirements?

- **A.** Use the job's matrix view of run history and configure an expected completion time on the job
- **B.** Open the Spark UI for the most recent run only
- **C.** Query `DESCRIBE HISTORY` on the job's output table
- **D.** Increase task retries so that slow runs are restarted

SECTION 7 Governance and Security — Exam weight 15%

Question 20

Objective: *Differentiate between managed and external tables in Unity Catalog and perform basic operations (create, modify, delete, and convert between managed and external tables) on them.*

With default settings, an engineer runs `DROP TABLE` on (1) the managed table `main.sales.orders_m` and (2) the external table `main.sales.orders_x`, which is stored at `s3://acme-ext/orders_x/`.

Which statement is correct?

- **A.** The data files for both tables are deleted immediately
- **B.** The managed table can be recovered with `UNDROP` for 7 days by default, after which its data files are purged; the external table's files are not deleted by Unity Catalog
- **C.** The managed table's files are kept indefinitely, and the external table's files are deleted
- **D.** Neither table can be dropped until its data files are removed manually

Question 21

Objective: *Configure access controls using the UI and SQL by applying GRANT, REVOKE, and DENY privileges to principals (users, groups, and service principals) at appropriate levels of the security hierarchy.*

The group `analysts` was granted `SELECT ON TABLE main.finance.revenue`, but its queries still fail with a permission error. No other grants exist for the group.

Which minimal additional grants are required?

- **A.** `GRANT USE CATALOG ON CATALOG main TO analysts` and `GRANT USE SCHEMA ON SCHEMA main.finance TO analysts`
- **B.** `GRANT ALL PRIVILEGES ON CATALOG main TO analysts`
- **C.** `GRANT MODIFY ON TABLE main.finance.revenue TO analysts`
- **D.** Transfer ownership of the schema `main.finance` to `analysts`

Question 22

Objective: *Understand Unity Catalog ABAC policies to centrally control row-level filtering and column masking for sensitive data.*

The catalog `prod` has hundreds of tables across many schemas. Every column tagged with the governed tag `pii` must be masked for everyone except the group `pii_readers`. Tables added and tagged later must be covered automatically.

Which approach meets these requirements with the least ongoing effort?

- **A.** Run `ALTER TABLE ... ALTER COLUMN ... SET MASK` on every PII column in every table
- **B.** Create an ABAC column mask policy at the catalog level that matches the governed tag `pii` and excludes `pii_readers`
- **C.** Create a dynamic view over every table and grant access only to the views
- **D.** Revoke SELECT on all tables from everyone except `pii_readers`

ANSWERS

Answer Key

Answers and explanations

#	Answer	Section
1	**B**	D1
2	**B**	D1
3	**A**	D2
4	**B**	D2
5	**C**	D2
6	**A**	D2
7	**B**	D3
8	**A**	D3
9	**B**	D3
10	**B**	D3
11	**B**	D4

#	Answer	Section
12	**A**	D4
13	**A**	D4
14	**A**	D5
15	**A**	D5
16	**A**	D5
17	**B**	D6
18	**B**	D6
19	**A**	D6
20	**B**	D7
21	**A**	D7
22	**B**	D7

REVIEW

Explanations

Rationale, distractors, and sources

Question 1 Section 1

Answer: B Use `DESCRIBE HISTORY` to identify the MERGE version, user, and timestamp, then run `RESTORE TABLE ... TO VERSION AS OF` the version before the MERGE

Why: `DESCRIBE HISTORY` returns the operation, user, and timestamp for each write, which covers the audit need. `RESTORE TABLE ... TO VERSION AS OF` returns the table to an earlier version through Delta time travel. The MERGE was only one day ago, which is well within the default 7-day `deletedFileRetentionDuration`, so the old files still exist.

Distractors: A: after VACUUM deletes the files, you can no longer restore to versions that reference them, so this destroys the recovery path. C: lineage shows dependencies and does not roll data back. D: a clone of the current table copies the bad state.

Source:
https://docs.databricks.com/aws/en/delta/history

Question 2 Section 1

Answer: B Serverless compute for jobs

Why: With serverless compute for jobs, Databricks manages the compute, and autoscaling and Photon are enabled automatically. It requires a Unity Catalog-enabled workspace and workloads that support standard access mode, and both conditions are met here.

Distractors: A: you pay for idle time, and all-purpose compute is meant for interactive work. C: fixed sizing and manual instance selection mean ongoing configuration work and no automatic scaling. D: SQL warehouses run SQL workloads, not general PySpark notebook code.

Source:
https://docs.databricks.com/aws/en/jobs/run-serverless-jobs

Question 3 Section 2

Answer: A COPY INTO skips files it has already loaded, even if they were modified; use `COPY_OPTIONS ('force' = 'true')` for the reload

Why: COPY INTO is idempotent. The docs say files that have already been loaded are skipped "even if the files have been modified since they were loaded." Setting `'force' = 'true'` turns idempotency off and loads the files again. The rows from the original bad files are still in the table, so delete them first or the table will contain duplicates.

Distractors: B: modification time does not matter, and modified files are still skipped. C: `mergeSchema` handles schema evolution, not reloading. D: checkpoint locations belong to Auto Loader and Structured Streaming, not COPY INTO.

Source:
https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into

Question 4 Section 2

Answer: B The stream fails with an `UnknownFieldException` after adding the new column to the schema, and restarting the stream resumes with the updated schema

Why: When no schema is provided, the default mode is `addNewColumns`. In that mode the stream fails with `UnknownFieldException` after Auto Loader adds the new columns to the schema, and a restart resumes processing with the updated schema. Databricks recommends Lakeflow Jobs with retries so the restart happens automatically.

Distractors: A: this is not the default behavior. C: this describes `rescue` mode. D: `addNewColumns` does not change existing column types; that is what `addNewColumnsWithTypeWidening` does.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/schema

Question 5 Section 2

Answer: C A Lakeflow Connect managed connector for Salesforce

Why: Lakeflow Connect managed connectors create ingestion pipelines that are governed by Unity Catalog, run on serverless compute, and use efficient incremental reads and writes. Salesforce is one of the supported SaaS sources, so this meets every requirement with the least custom code.

Distractors: A: custom code the team would have to maintain. B: adds an export step and a second system. D: full reloads are not incremental.

Source:
https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/

Question 6 Section 2

Answer: A Switch Auto Loader to file notification mode using file events

Why: The docs say file notification mode with file events is more performant and scalable than directory listing, and Databricks recommends it for most workloads. Directory listing has to list the input path again to find new files, and that cost grows with file volume.

Distractors: B: limits how many files each batch processes but does not make discovery cheaper. C: COPY INTO is not designed for millions of files per day at this cadence. D: schema mode has no effect on file discovery.

Source:
https://docs.databricks.com/aws/en/ingestion/cloud-object-storage/auto-loader/file-detection-modes

Question 7 Section 3

Answer: B `df.dropDuplicates(["order_id"])`

Why: `dropDuplicates` with a subset of columns removes rows that share the same values in those columns, which leaves one row per `order_id`. If you need to keep a specific row, such as the latest `ingest_ts`, use a window with `row_number()` instead, because `dropDuplicates` does not guarantee which row it keeps.

Distractors: A: `distinct()` compares all columns, and the duplicates differ in `ingest_ts`, so they are all kept. C: returns only counts and loses the other columns. D: removes rows with null IDs, not duplicates.

Source:
https://docs.databricks.com/aws/en/pyspark/basics
https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.dropDuplicates.html

Question 8 Section 3

Answer: A `transactions.join(broadcast(stores), "store_id")`

Why: A broadcast hint sends the small table to every executor, so the large side is joined in place and never shuffled. At 80 MB the dimension is larger than the static `spark.sql.autoBroadcastJoinThreshold` default (10 MB) and the AQE runtime threshold `spark.databricks.adaptive.autoBroadcastJoinThreshold` (30 MB), so Spark will not broadcast it automatically. The explicit hint is needed.

Distractors: B: a cross join is a Cartesian product, which is enormous here. C: a union stacks rows and does not join them. D: more partitions still means a full shuffle.

Source:
https://docs.databricks.com/aws/en/optimizations/aqe

Question 9 Section 3

Answer: B A materialized view

Why: Materialized views cache their results and update them as the source tables change. Where possible they refresh incrementally by merging only new or modified data. The docs name keeping a BI dashboard up to date with minimal query latency as a key use case.

Distractors: A/D: views compute results on demand every time they are queried. C: streaming tables are designed for append-only sources; for sources with updates or deletes, the docs point to `AUTO CDC`.

Source:
https://docs.databricks.com/aws/en/views/materialized
https://docs.databricks.com/aws/en/ldp/streaming-tables

Question 10 Section 3

Answer: B `CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW`

Why: `ON VIOLATION DROP ROW` drops invalid records before they are written, and the count of dropped records is logged with the dataset metrics. The pipeline keeps running.

Distractors: A: the default action is warn, which writes invalid rows to the target. C: stops the update and records no metrics. D: a Delta CHECK constraint makes the write fail rather than dropping rows.

Source:
https://docs.databricks.com/aws/en/ldp/expectations

Question 11 Section 4

Answer: B A table update trigger on `silver.orders` with *Wait after last change* set to about 15 minutes

Why: A table update trigger runs the job when the monitored table is updated. *Wait after last change* delays the run, and any additional update during that window restarts the timer, so the job waits until the multi-commit batch is finished. Unity Catalog managed Delta tables are supported.

Distractors: A: still time-based and adds unnecessary latency. C: file arrival triggers watch external locations and volumes for new files, not table commits. D: wastes compute and does not wait for the data.

Source:
https://docs.databricks.com/aws/en/jobs/trigger-table-update
https://docs.databricks.com/aws/en/jobs/file-arrival-triggers

Question 12 Section 4

Answer: A A For each task with Inputs `{{tasks.get_regions.values.regions}}` and Concurrency set to 5

Why: The For each task runs a nested task in a loop and passes different parameters to each iteration. Its inputs can come from task values using `{{tasks..values.}}`, and the Concurrency setting (default 1) controls how many iterations run in parallel.

Distractors: B: hard to maintain and ignores the dynamic list. C: If/else is for branching, not looping. D: creates 25 separate job runs and is not native loop orchestration.

Source:
https://docs.databricks.com/aws/en/jobs/for-each

Question 13 Section 4

Answer: A An If/else condition task with `{{tasks.validate.values.bad_row_count}} > 0`, with `quarantine_report` on the true branch and `publish_gold` on the false branch

Why: The If/else condition task adds boolean logic to the task graph. Operands can reference task values, and >, >=, <, <= perform numeric comparison, while == and `!=` perform string comparison.

Distractors: B: retries re-run failed tasks and do not branch. C: *Run if* is based on the status of upstream tasks, not on a value. D: For each is for looping.

Source:
https://docs.databricks.com/aws/en/jobs/if-else

Question 14 Section 5

Answer: A Declare a custom variable `catalog` with a default, reference `${var.catalog}` in the job, and override it under `targets..variables`

Why: Custom variables are declared in the `variables` mapping, referenced with `${var.}`, and can be set per target in the `variables` mapping under `targets`. Values can also be passed with `--var` or `BUNDLE_VAR_` environment variables. Precedence, highest first: `--var`, then environment variable, then override file, then target, then default.

Distractors: B/D: manual, duplicated, and error-prone. C: dev would point at prod data.

Source:
https://docs.databricks.com/aws/en/dev-tools/bundles/variables

Question 15 Section 5

Answer: A `databricks bundle validate -t prod`, then `databricks bundle deploy -t prod`

Why: `bundle validate` checks the bundle configuration for the chosen target without deploying anything. `bundle deploy` then deploys the resources to that target. `bundle run` is used afterward to start a deployed job or pipeline.

Distractors: B: runs before validating and deploying. C: `init` creates a new bundle from a template. D: manual steps that skip bundles entirely.

Source:
https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands
https://docs.databricks.com/aws/en/dev-tools/bundles/

Question 16 Section 5

Answer: A Set `mode: development` on the dev target

Why: `mode: development` adds the prefix `[dev ${workspace.current_user.short_name}]` to resource names, pauses all schedules and triggers, marks pipelines as development, and allows concurrent runs.

Distractors: B: production mode adds validations such as Git branch checks and does not prefix or pause anything. C: Git folders isolate code, not deployed resource names. D: manual edits that are easy to forget.

Source:
https://docs.databricks.com/aws/en/dev-tools/bundles/deployment-modes

Question 17 Section 6

Answer: B Increase the number of shuffle partitions (or set `spark.sql.shuffle.partitions=auto`) so that each partition fits in memory

Why: Max is close to the 75th percentile, so the stage is not skewed. The docs flag skew when Max is much higher than the 75th percentile. The spill stats point to partitions that are too large for memory, and "too few shuffle partitions" is a listed cause of memory problems. Databricks recommends `spark.sql.shuffle.partitions=auto`.

Distractors: A: salting fixes skew, and there is none here. C: fewer partitions makes each one larger and worsens spill. D: AQE helps with partition sizing and skew.

Source:
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-page
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-memory-issues
https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-io

Question 18 Section 6

Answer: B Migrate to liquid clustering, for example `CLUSTER BY AUTO` with predictive optimization enabled

Why: Liquid clustering replaces partitioning and Z-order. You can redefine clustering keys without rewriting existing data, and subsequent `OPTIMIZE` runs apply the new keys incrementally. `CLUSTER BY AUTO` (DBR 15.4 LTS and above) chooses keys based on query patterns and requires predictive optimization.

Distractors: A: more partition columns means even more small files. C: liquid clustering cannot be combined with partitioning or ZORDER on the same table. D: ZORDER requires ongoing manual tuning and does not adapt.

Source:
https://docs.databricks.com/aws/en/delta/clustering

Question 19 Section 6

Answer: A Use the job's matrix view of run history and configure an expected completion time on the job

Why: The matrix view shows the job's run history, including each task, and the height of each bar shows the run's duration, so trends are visible at a glance. If an expected completion time is configured, the matrix view and the runs list show a warning when a run exceeds it.

Distractors: B: shows a single run with no historical baseline. C: shows table writes, not job durations. D: retries do not help with monitoring.

Source:
https://docs.databricks.com/aws/en/jobs/monitor

Question 20 Section 7

Answer: B The managed table can be recovered with `UNDROP` for 7 days by default, after which its data files are purged; the external table's files are not deleted by Unity Catalog

Why: Managed tables are recoverable for 7 days by default, and this window can be set from 0 hours to 30 days at the catalog or schema level. After it ends, an asynchronous purge deletes the files. For external tables, Unity Catalog does not delete the underlying data, and you must remove the files yourself.

Distractors: A, C, and D all contradict the documented behavior.

Source:
https://docs.databricks.com/aws/en/tables/managed
https://docs.databricks.com/aws/en/tables/external

Question 21 Section 7

Answer: A `GRANT USE CATALOG ON CATALOG main TO analysts` and `GRANT USE SCHEMA ON SCHEMA main.finance TO analysts`

Why: To read a table, a user needs SELECT on the table, USE CATALOG on the parent catalog, and USE SCHEMA on the parent schema. This is least-privilege access.

Distractors: B and D grant far more than needed. C: MODIFY is for writes and does not fix the missing USE privileges.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/privileges

Question 22 Section 7

Answer: B Create an ABAC column mask policy at the catalog level that matches the governed tag `pii` and excludes `pii_readers`

Why: ABAC policies use governed tags and can be attached at the catalog or schema level (or the metastore level, in Beta). They are evaluated dynamically, so one policy enforces consistent masking on every matching column, including columns tagged later.

Distractors: A/C: configured table by table, and new tables are missed. D: blocks the whole table instead of masking specific columns.

Source:
https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/

WARNING Q7: the Databricks doc confirms only `distinct()`. The subset behavior of `dropDuplicates()` comes from the Apache Spark API reference, which is linked. Q8: the static 10 MB `spark.sql.autoBroadcastJoinThreshold` default is not on the Databricks AQE page; confirm it in the Spark SQL performance tuning docs.